## Building a training set with feature tables

Recall from *Chapter 5* that the **Databricks Feature Engineering** (**DFE**) client enables us to build feature tables that serve as central repositories for features, or variables, that can be used in downstream projects. A DFE `training_set` is the combination of your training data and your features via a `FeatureLookup` object. Training sets enable you to create new training datasets with the same specifications quickly. This can reduce duplicative work by making it easy to share feature tables. For example, many people in an organization may need the same customer features (such as age, country of residence, or number of transactions for the last year) for their various projects. However, without an agreed-upon repository for these features, each person could be using different source data and calculations to build them, resulting in multiple copies of inconsistent features being used for each project. Using training sets and feature tables helps avoid these inconsistencies and prevents the inefficient re-creation of the same feature. A feature table can be created and used by everyone in an organization.

In *Chapter 5*, we looked at how to use the DFE API to build and store features. The Streaming Transactions and Favorita Store Sales projects now have engineered features that can augment the data used to train models, but how do we easily bring together these new features to create a training dataset? We use the DFE feature lookup functionality. The DFE API function `create_training_set()` has a parameter called `feature_lookups`, where you can specify a list of feature tables to include in your training set. This list will reference the `FeatureLookup` objects. For each `FeatureLookup`, you must specify the name of the feature table you want to bring into your training dataset, and the `lookup_key` (one or more) used to match the right feature values to the corresponding records in your training set. You can think of the `lookup_key` as a join key. Note that you can specify a subset of features from the feature table to include in your training set, rather than including every column, by including the feature names in the `FeatureLookup's feature_names` argument.

We also briefly mentioned point-in-time lookups in *Chapter 5*, or features that must be joined with a timestamp. You can use point-in-time lookups by storing time-dependent features in a time series feature table and including a `timestamp_lookup_key` in the `FeatureLookup` object. *Figure 6.1* is an example of creating a training set spec using a point-in-time look-up.